

Problem Statement – Ai extension is not supporting Playwright using type script

Objective - To create a feature file in AI Extension with Playwright and typescript

Login

Email Address

Password

☐ Stay logged in for the next 8 hours ⓘ [Forgot password?](#)

Login

or

Create Account

Newsletter registration

Sign up for the newsletter and receive expert knowledge and exclusive offers directly in your mailbox.

To the newsletter registration

Contact

Do you have any questions or suggestions? Let us know and get in touch with us via the contact form.

To the contact form

Information

- FAQ & Shop Features
- Imprint
- Disclaimer
- Terms of Use
- Terms and Conditions
- Data Protection Notice
- Technical Documents
- Cooke Settings

Reset Chat

⚠️ playwright combination is not yet supported. Only JavaSelenium is currently available.

Feature: Login functionality for South

Scenario Outline: Successful login w

Given I open the login page

When I type "email" into the fna

And I type "password" into the P

And I click the login button

Then I should be logged in success

Examples:

email	pa
arun.kumar@tech.com	Ar
sita.patel@info.com	Si
kumar.ra@services.in	Ra

Scenario Outline: Unsuccessful login

Given I open the login page

When I type "email" into the fna

And I type "password" into the P

And I click the login button

Then I should see an error message

Examples:

email	pa
invalid@domain.com	W-
arun.kumar@tech.com	W-
sita.patel@info.com	12

Scenario Outline: Stay logged in tog

Given I open the login page

When I type "email" into the fna

And I type "password" into the P

And I toggle the stay logged in fo

And I click the login button

Then I should be logged in success

Examples:

email	pa
arun.kumar@tech.com	Ar
sita.patel@info.com	Si

Scenario: Click forgot password link

Given I open the login page

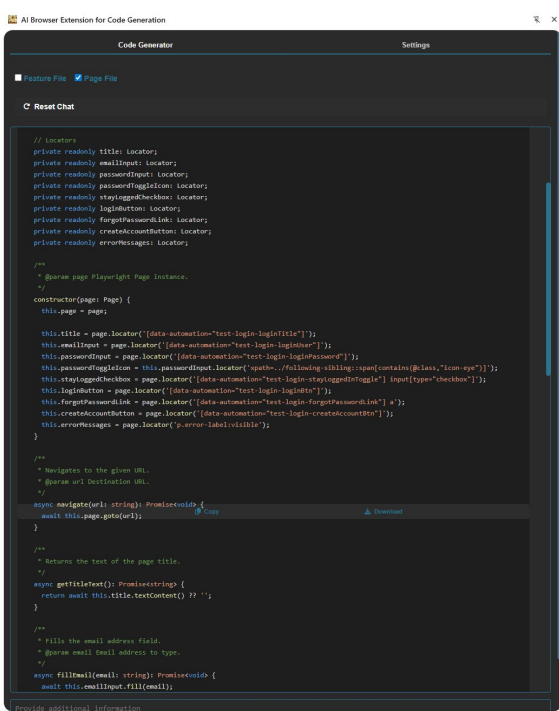
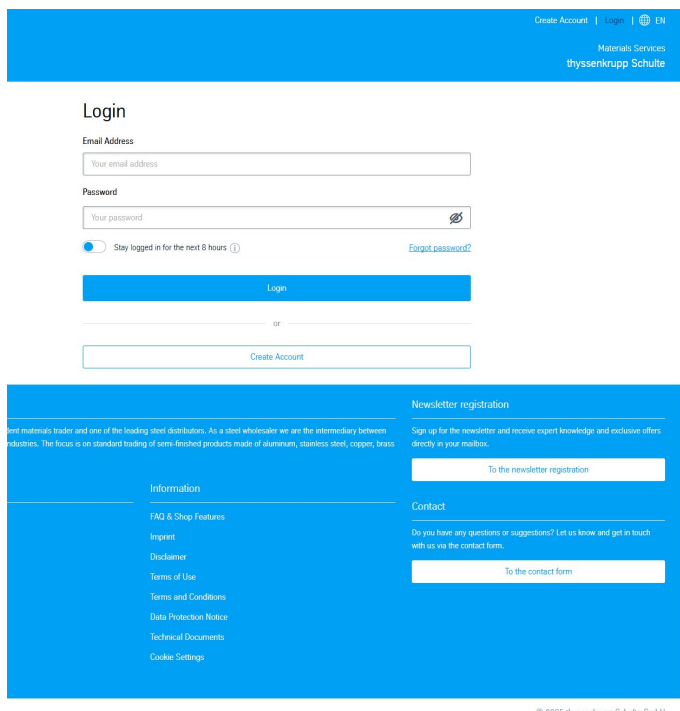
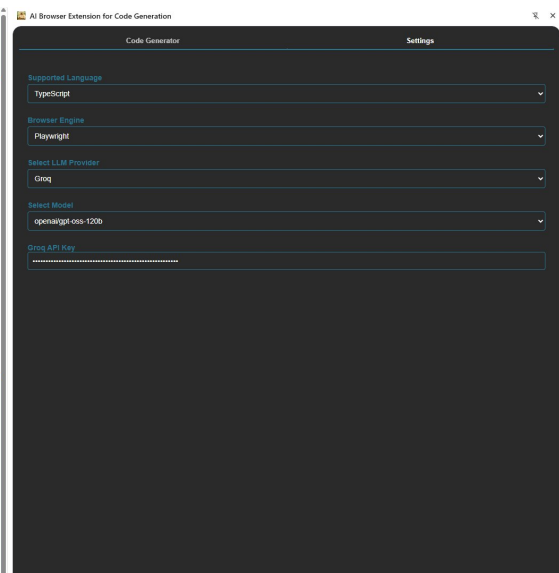
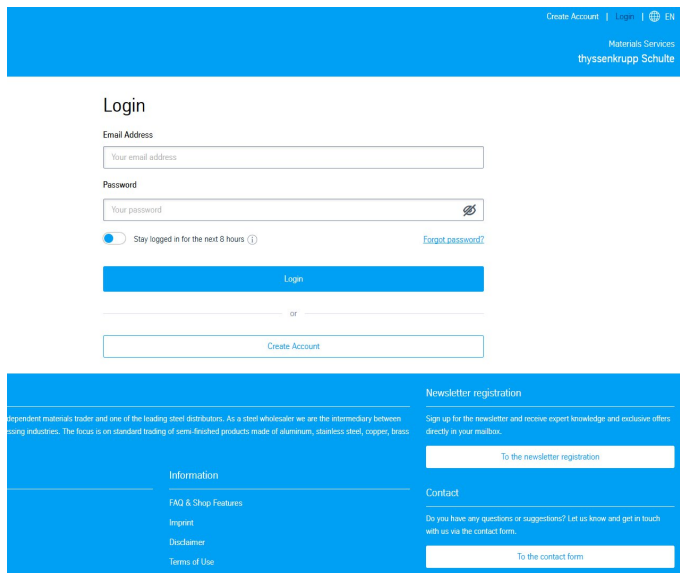
Prompt for vibe coding -

Act as a 9+ year full stack developer

[SRICTLY] Read my entire workspace and understand how ai extension is implemented

[MANDATORY] Currently, my extension supports selenium using java, so I need my extension to support playwright using typescript

[MUST]Extension must create page files using playwright with typescript



Generated code -

import { Page, Locator } from '@playwright/test';

/**

* Page Object representing the Login page.

*

* Provides high-level actions and queries for interacting with the login form,

* including email/password entry, toggling the "stay logged in" option,

* and navigation to related pages.

*/

```
export default class LoginPage {
```

```
  private readonly page: Page;
```

```
  // Locators
```

```
  private readonly title: Locator;
```

```
  private readonly emailInput: Locator;
```

```
  private readonly passwordInput: Locator;
```

```
  private readonly passwordToggleIcon: Locator;
```

```
  private readonly stayLoggedInCheckbox: Locator;
```

```
  private readonly loginButton: Locator;
```

```
  private readonly forgotPasswordLink: Locator;
```

```
  private readonly createAccountButton: Locator;
```

```
  private readonly errorMessages: Locator;
```

```
  /**
```

```
   * @param page Playwright Page instance.
```

```
  */
```

```
  constructor(page: Page) {
```

```
    this.page = page;
```

```
    this.title = page.locator('[data-automation="test-login-loginTitle"]');
```

```
    this.emailInput = page.locator('[data-automation="test-login-loginUser"]');
```

```

    this.passwordInput = page.locator('[data-automation="test-login-loginPassword"]');

    this.passwordToggleIcon = this.passwordInput.locator('xpath=../following-sibling::span[contains(@class,"icon-eye")]');

    this.stayLoggedInCheckbox = page.locator('[data-automation="test-login-stayLoggedInToggle"] input[type="checkbox"]');

    this.loginButton = page.locator('[data-automation="test-login-loginBtn"]');

    this.forgotPasswordLink = page.locator('[data-automation="test-login-forgotPasswordLink"] a');

    this.createAccountButton = page.locator('[data-automation="test-login-createAccountBtn"]');

    this.errorMessages = page.locator('p.error-label:visible');
}

```

```
/**
```

```
 * Navigates to the given URL.
```

```
 * @param url Destination URL.
```

```
 */
```

```

async navigate(url: string): Promise<void> {
    await this.page.goto(url);
}

```

```
/**
```

```
 * Returns the text of the page title.
```

```
 */
```

```

async getTitleText(): Promise<string> {
    return await this.title.textContent() ?? "";
}

```

```
/**  
 * Fills the email address field.  
 * @param email Email address to type.  
 */  
async fillEmail(email: string): Promise<void> {  
    await this.emailInput.fill(email);  
}
```

```
/**  
 * Fills the password field.  
 * @param password Password to type.  
 */  
async fillPassword(password: string): Promise<void> {  
    await this.passwordInput.fill(password);  
}
```

```
/**  
 * Clicks the eye icon to toggle password visibility.  
 */  
async togglePasswordVisibility(): Promise<void> {  
    await this.passwordToggleIcon.click();  
}
```

```
/**  
 * Sets the "Stay logged in" checkbox to the desired state.  
 * @param enable `true` to check, `false` to uncheck. If omitted, the checkbox is  
toggled.
```

```
*/  
  
async setStayLoggedIn(enable?: boolean): Promise<void> {  
    const isChecked = await this.stayLoggedInCheckbox.isChecked();  
    if (enable === undefined) {  
        await this.stayLoggedInCheckbox.click();  
    } else if (enable !== isChecked) {  
        await this.stayLoggedInCheckbox.click();  
    }  
}
```

```
/**  
 * Clicks the Login button.  
 */
```

```
async clickLogin(): Promise<void> {  
    await this.loginButton.click();  
}
```

```
/**  
 * Returns whether the Login button is enabled.  
 */
```

```
async isLoginButtonEnabled(): Promise<boolean> {  
    return await this.loginButton.isEnabled();  
}
```

```
/**  
 * Clicks the "Forgot password?" link.  
 */
```

```
async clickForgotPassword(): Promise<void> {  
    await this.forgotPasswordLink.click();  
}
```

```
/**
```

```
 * Clicks the "Create Account" button.
```

```
 */
```

```
async clickCreateAccount(): Promise<void> {  
    await this.createAccountButton.click();  
}
```

```
/**
```

```
 * Retrieves all currently visible error messages on the page.
```

```
 */
```

```
async getVisibleErrorMessages(): Promise<string[]> {  
    const count = await this.errorMessages.count();  
    const messages: string[] = [];  
    for (let i = 0; i < count; i++) {  
        const text = await this.errorMessages.nth(i).innerText();  
        messages.push(text.trim());  
    }  
    return messages;  
}
```

```
/**
```

```
 * Waits for the login form to be visible.
```

```
 */
```

```
async waitForLoad(): Promise<void> {  
    await this.title.waitFor({ state: 'visible' });  
    await this.emailInput.waitFor({ state: 'visible' });  
    await this.passwordInput.waitFor({ state: 'visible' });  
}  
}
```